Chart the Invisible: Interactive Perception for Hidden-Target Exploration in GUI Agents

YONGXIANG HU, College of Computer Science and Artificial Intelligence, Fudan University, China
RUIYANG LIU, College of Computer Science and Artificial Intelligence, Fudan University, China
QINING HE, College of Computer Science and Artificial Intelligence, Fudan University, China
XUAN WANG, College of Computer Science and Artificial Intelligence, Fudan University, China
YANGFAN ZHOU, College of Computer Science and Artificial Intelligence, Fudan University, China

Despite rapid progress in command-line interface (CLI) tasks, recent LLM-based agents still struggle with reliable task execution through graphical user interfaces (GUIs). A key challenge is the limited content visible within a bounded viewport, especially on mobile devices, where many task-relevant targets remain hidden and can only be revealed through swipe interactions. Therefore, although most GUI agents support swipe as an action primitive, the lack of target visibility makes it difficult for them to decide when, where, and how to swipe to reveal the hidden content. Our quantitative study further shows that GUI agents incorrectly choose a click action in 47.8% of cases that require swipe exploration. To bridge this gap, this paper proposes equipping GUI agents with scrollable-region perception to support hidden-area exploration before action selection. To this end, we trained the first multimodal model for scrollable-region perception on a manually collected dataset of 667 UI screens, called `Hagasu`. By correctly identifying 78.5% of scrollable regions, experiments across nine GUI agents show that integrating `Hagasu` improves the mean end-to-end completion rate from 64.8% to 82.7% on explore-dependent tasks. To facilitate follow-up research, we release the swipe dataset and the source code of `Hagasu`.

CCS Concepts: • **Computing methodologies** → **Intelligent agents**; *Computer vision*; • **Human-centered computing** → *Graphical user interfaces.*

Additional Key Words and Phrases: LLM Agents, GUI Automation, Scrollable Perception, Swipe Planning, Vision-Language Models

1 Introduction

Although large language models (LLMs) have made surprising progress on command-line interface (CLI) tasks [48, 49], reliable task execution through graphical user interfaces (GUIs) remains challenging [20, 46]. Beyond the well-known challenge of interpreting visual semantics, the limited content visible within a bounded viewport constitutes a key bottleneck for GUI agents. Specifically, to preserve visual appeal, UI designs constrain element density and thereby prevent visual clutter [34]. Modern UIs therefore widely use scrollable containers, such as lists and carousels, to accommodate additional functionality and content beyond the current view, requiring scrolling to reveal them [29]. At every step, GUI agents are thus forced to decide under uncertainty whether to pick a visible target or explore hidden areas.

This dilemma is particularly acute in mobile GUIs. Limited screen size leaves many UI elements outside the current viewport, accessible only through swipe interactions. Although most GUI agents already support swipe as an action primitive [25, 30, 33, 41, 54], recent research shows that reliably completing mobile tasks remains challenging [20, 32]. To quantify this difficulty, this paper conducts a study of 7 GUI agents across 218 tasks and finds that the rate of correctly planned swipe actions is 47.3 percentage points lower than that of click actions. Further analysis of 83.73% failed

Authors' Contact Information: Yongxiang Hu, College of Computer Science and Artificial Intelligence, Fudan University, Shanghai, China; Ruiyang Liu, College of Computer Science and Artificial Intelligence, Fudan University, Shanghai, China; Qining He, College of Computer Science and Artificial Intelligence, Fudan University, Shanghai, China; Xuan Wang, College of Computer Science and Artificial Intelligence, Fudan University, Shanghai, China; Yangfan Zhou, College of Computer Science and Artificial Intelligence, Fudan University, Shanghai, China.

tasks shows that a major cause of failure is clicking an incorrect visible target when the correct target is still hidden. These findings highlight the need to help agents recognize opportunities for hidden-area exploration rather than force a choice among visible elements.

Therefore, this paper focuses on equipping GUI agents with scrollable-region perception, providing explicit cues about where hidden-area exploration is possible. Achieving this goal involves two challenges. First, scrollable regions lack clear visual indicators, requiring an understanding of UI structure and semantics to identify them. To address this challenge, this paper builds on the UI understanding capabilities of a pretrained vision-language model (VLM) and fine-tunes it on 540 manually collected and annotated UI screens, to develop `Hagasu` for scrollable region perception. Second, improving a specific capability of a generalist GUI agent may compromise its broader capabilities. This paper therefore implements `Hagasu` as a non-intrusive plug-in that modifies only the GUI screenshot and prompt presented to the agent: it marks predicted scrollable regions on the screenshot and adds textual cues for swipe exploration.

Extensive experiments evaluate `Hagasu` in scrollable-region perception, swipe planning, and end-to-end task completion. Comparisons and ablations across eight configurations demonstrate the effectiveness of `Hagasu`'s training design for scrollable-region perception, with the full configuration achieving the highest recall of 78.46%. In offline swipe-planning experiments with nine VLMs, `Hagasu` raises mean swipe prediction recall from 46.02% to 53.87%. And it decreases click prediction recall by only 1.67 percentage points. Finally, online experiments show that integrating `Hagasu` into existing GUI agents increases mean end-to-end task success rate from 64.81% to 82.72%. To facilitate follow-up research, we release the scrollable-region dataset and the source code of `Hagasu`.

We summarize the contributions of this paper as follows.

- We systematically identify off-screen content as a critical, understudied bottleneck that partially explains the performance gap between GUI and CLI agents.
- We build the first dataset of scrollable UI regions and open-source `Hagasu`, a multimodal model that equips GUI agents with scrollable-region perception for proactive hidden-area exploration.
- Comprehensive experiments demonstrate the effectiveness of scrollable-region perception in improving GUI agents' task completion on explore-dependent tasks.

The rest of the paper is organized as follows. Section 2 presents the background and the preliminary study motivating this work. Section 3 describes the data collection and annotation, model fine-tuning, and plug-in design of `Hagasu`. Sections 4 and 5 present the experimental design and analyze the results, respectively. Section 6.2 discusses threats to validity, and Section 7 reviews related work. Finally, Section 8 concludes the paper.

2 Background

This section explains the decision pattern underlying ordinary click interactions and why hidden targets require a different course of action. We first describe greedy interaction as selecting a task-relevant component from the current viewport. We then explain why agents need swipe-based exploration when the correct target is hidden outside that viewport. Finally, we present a preliminary study comparing click and swipe planning across different interaction scenarios.

2.1 Greedy Interaction Patterns for Visible Targets

GUI agents commonly follow an observation, decision, and execution loop, selecting actions based on the current interface, task instruction, and interaction history [32, 41, 51, 54]. For a click action, this decision involves selecting an element in the current viewport and clicking it. We describe this pattern as *greedy interaction*: choosing the visible element that appears most likely to advance the

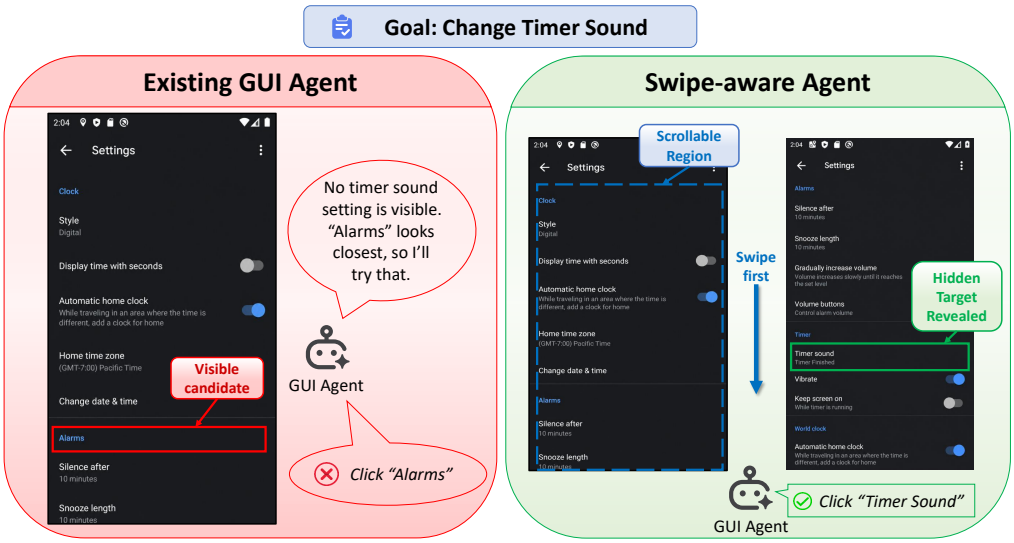


Fig. 1. Motivating example of swipe planning for a hidden target.

task. Here, greedy characterizes the choice among currently visible candidates, even when guided by an explicit plan or interaction history.

This strategy works when the current viewport contains the correct target, reducing click planning to semantic matching and coordinate grounding. For example, an agent can fulfill a request to open Settings by identifying and clicking the visible Settings button. Existing GUI grounding methods support this process by mapping task descriptions to visible UI elements and their locations [14, 16, 36, 44, 53].

However, as shown in Figure 1, when the correct target is hidden, even the most relevant visible element may be an incorrect choice. An agent that continues to select among visible candidates may therefore force a wrong click instead of reaching the required target, as shown in Figure 1. The decision must then extend beyond which element to click to whether exploration is needed to reveal the hidden target.

2.2 Hidden Area Exploration via Swipe Interaction

As mobile apps offer more functionality, their increasingly dense interfaces often contain more content than the screen can display at once. Task-relevant targets may be partially clipped or completely hidden within full-page or local scrollable regions, such as lists and carousels [32, 33, 47]. When the target lies beyond the visible content, the agent must swipe to reveal it and then reassess the available interaction targets.

Planning this exploration requires the agent to decide when, where, and how to swipe before the target becomes visible. For example, finding a category hidden in a horizontal carousel requires swiping that carousel rather than clicking another visible category. The agent must infer the region's function and relevance to the task, locate its boundaries, and choose a suitable swipe direction and extent. Multiple or nested scrollable regions complicate this decision, as swiping the wrong region may reveal content unrelated to the task.

Existing GUI agents already support swipe as an action primitive [30, 41, 54]. However, this support alone does not ensure that an agent can recognize when exploration is necessary or select

the appropriate region to explore. This distinction motivates a quantitative comparison between click planning with visible targets and swipe planning for hidden-area exploration.

2.3 Preliminary Study of Scrollable Perception

To examine the difficulty of hidden-area exploration compared with visible-target selection, we conduct a preliminary interaction study. The study compares click and swipe planning by prompting GUI agents to locate a specific UI element, either visible or hidden, and assessing whether they select the correct next interaction based on the current screenshot.

2.3.1 Study setup. Both general-purpose VLM GUI agents and specialized GUI models are selected for this study. Specifically, general-purpose VLM GUI agents include Qwen3-VL [15] and GPT-5.4 [6]. As for the specialized GUI models, we include GUI-Owl [52], UI-TARS [30], GUI-G^2 [38], UI-Venus 1.5 [39] and MAI-UI [55]. To control for differences in input, all agents receive the same current screenshot and target element description.

2.3.2 Task configuration. Using an Android phone connected through ADB, we manually collected 218 UI interactions from 55 applications, including 36 Chinese-language and 19 English-language applications. These interactions include 192 swipe cases requiring exploration to reveal or reach the target component and 26 click cases with visible targets as the reference condition.

Since scrollable regions vary in appearance, we categorize these swipe interactions by direction (vertical or horizontal), scope (full-screen or local), and target visibility (partially visible or completely invisible). Specifically, full-screen scrolling refers to scrollable regions occupying more than 75% of the screen, while all remaining cases are classified as local swiping. As for visibility, we classify a target as completely invisible when less than 25% of its area is visible, and as partially visible when 25% to 75% is visible. The upper bound reflects the likelihood that targets with more than 75% of their area visible can already be interacted with directly.

2.3.3 Evaluation metrics. Instead of running each agent end-to-end on complete tasks, we focus on whether it can produce the *correct next interaction* to facilitate comparison across agents. Although swipe actions involve multiple parameters, including position, direction, and distance [33], this study focuses on position and direction, since the remaining parameters are less relevant to scrollable perception. We then report the proportion of correctly planned interactions in each task category to compare click and swipe planning.

2.3.4 Results and Implications. Table 1 shows a substantial gap between visible-target clicking and hidden-area exploration. Weighting the eight swipe categories by their actual sample counts and averaging across the evaluated agents, the mean success rate is 90.1% for click cases and 42.9% for swipe cases, a decrease of 47.3 percentage points. Moreover, agents' strong click capabilities do not translate into reliable swipe planning. For example, GUI-Owl, UI-Venus 1.5, and MAI-UI each achieve 100% on click cases, but their sample-weighted swipe success rates are only 42.2%, 35.4%, and 40.6%, respectively.

Direction is the clearest source of variation: performance averages 57.4% for vertical swipes but only 25.3% for horizontal swipes. This pattern is consistent with the vertical scrolling convention used for UI lists [7]. Moreover, target visibility is associated with a substantial difference in planning accuracy: completely invisible targets achieve a higher weighted average than partially visible targets (47.2% versus 36.8%). The gap between full-screen and local swiping is the smallest (44.6% versus 41.3%), suggesting that region size is not the primary bottleneck in current agents' GUI understanding.

Further analysis reveals a dominant failure mode: in 83.73% of failed swipe tasks, agents decided to click on an incorrect visible target instead of exploring the correct but hidden region. Only

Table 1. Success rates (%) of representative GUI agents on click and swipe step-level interaction tasks. Part-Inv. and Inv. denote partially visible and completely invisible targets, respectively.

Action	Scope	Visibility	General-purpose Models		Specialized GUI Models				
			Qwen3-VL	GPT-5.4	GUI-Owl	UI-TARS	GUI-G^2	UI-Venus 1.5	MAI-UI
Click	–	Visible	96.2	96.2	100.0	50.0	88.5	100.0	100.0
Vertical Swipe	Full-screen	Part-Inv.	71.4	47.6	61.9	71.4	66.7	66.7	52.4
		Inv.	76.5	61.8	61.8	64.7	47.1	64.7	61.8
	Local	Part-Inv.	42.9	47.6	47.6	52.4	38.1	33.3	42.9
		Inv.	62.1	69.0	58.6	48.3	55.2	51.7	58.6
Horizontal Swipe	Full-screen	Part-Inv.	11.8	11.8	11.8	5.9	29.4	5.9	17.6
		Inv.	17.6	23.5	17.6	23.5	23.5	5.9	11.8
	Local	Part-Inv.	23.8	38.1	19.0	38.1	19.0	4.8	19.0
		Inv.	37.5	59.4	34.3	28.1	43.8	21.9	34.4

fewer than 30% of failures involve selecting the wrong scrollable region or swipe direction. These findings reveal a strong tendency for LLM-based GUI agents to settle for seemingly suitable visible components rather than explore hidden areas.

These results indicate that merely including swipe in an agent's action space is far from sufficient for reliable GUI interaction. Agents still struggle to determine whether exploration is necessary and, if so, where and in which direction to swipe. These findings motivate our subsequent investigation into prompting agents with scrollable-region information to encourage exploration and reduce incorrect clicks.

3 Method

This section presents Hagasu, a scrollable-region perception module that supports hidden-area exploration through non-intrusive augmentation of GUI agents' screenshots and prompts.

3.1 Overview

As shown in Figure 2, the Hagasu workflow consists of dataset construction, model fine-tuning, and agent integration. First, we collect Android screenshots and accessibility XML through ADB and manually verify and refine XML-derived region candidates to construct screenshot–region pairs. Second, we build on Qwen3.5-9B's UI understanding capabilities through two-stage LoRA fine-tuning, enabling it to predict all scrollable regions as a JSON list of bounding boxes. Third, when integrated into a GUI agent, Hagasu marks its predicted regions on the screenshot at each step and supplements the agent's prompt with cues for swipe-based exploration. The base agent combines these cues with the task and interaction history to plan its next action, retaining control over whether and how to click or swipe.

3.2 Dataset Construction

3.2.1 Data Acquisition and Scrollable-Region Annotation. Although public GUI interaction datasets such as Android in the Wild (AITW) [33] and GUIOdyssey [27] provide screenshots and action trajectories, they lack complete bounding-box annotations of all scrollable regions in each screenshot. As shown in Table 2, even existing first-tier models struggle to identify scrollable regions accurately, making automatic annotation infeasible. We therefore manually collect and annotate UI screenshots to provide reliable supervision for scrollable-region perception training.

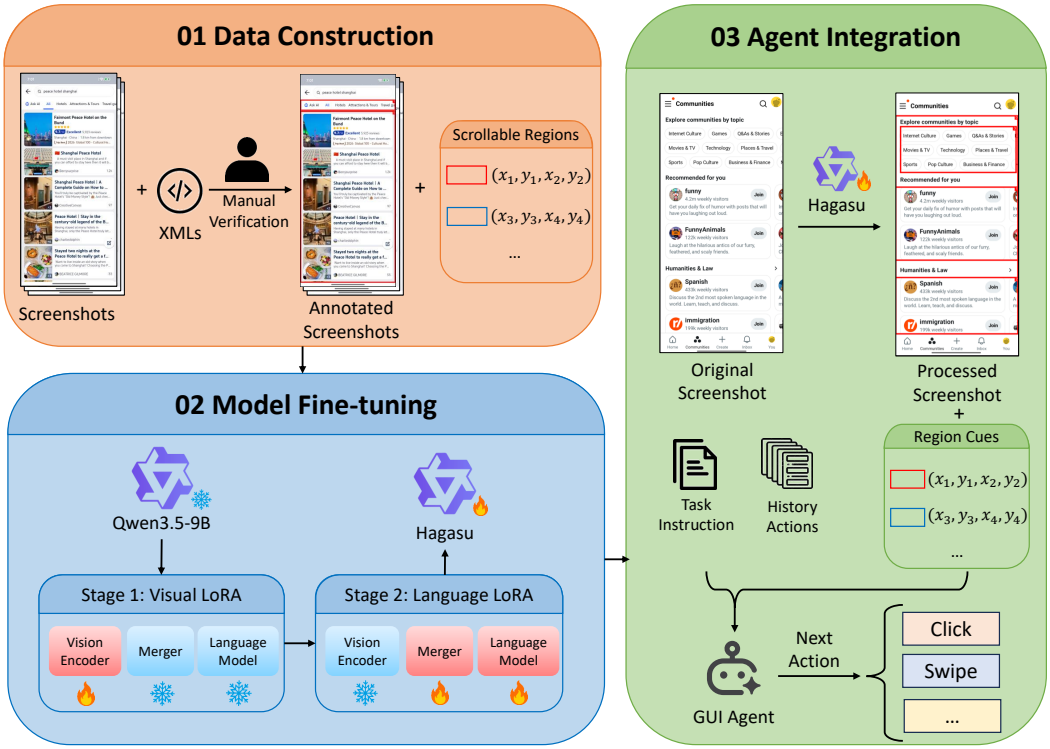


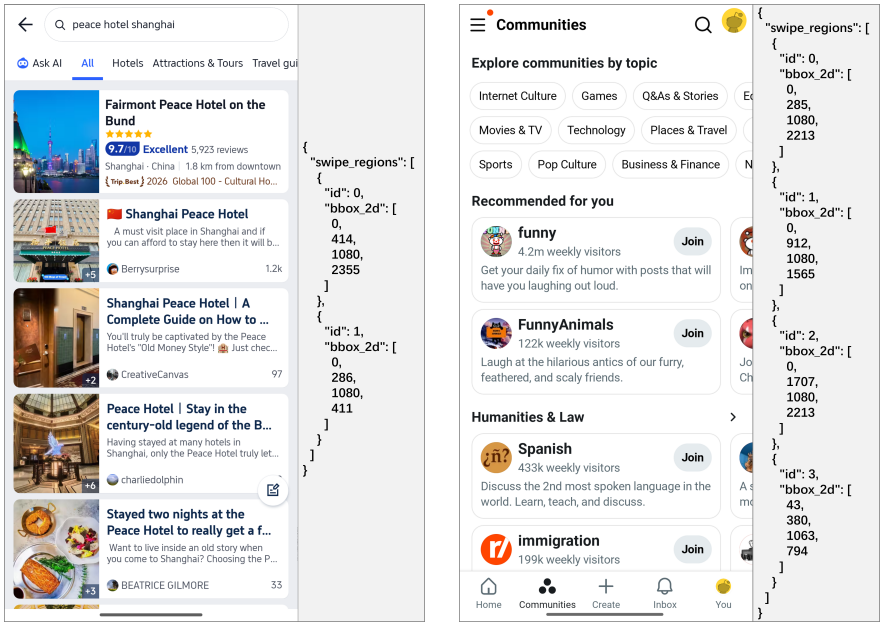
Fig. 2. Overview of Hagasu.

49 apps are selected from Google Play's popular app rankings [2], including 33 Chinese- and 16 English-language apps. For each app, we manually navigate its interface and save screenshots containing scrollable regions. To reduce annotation effort, we also capture the corresponding accessibility XML for each screenshot through Android Debug Bridge (ADB) [1], since the UI component attributes in the XML can support preliminary region annotation [13]. Specifically, we parse each XML node's bounds attribute into a screen-coordinate rectangle, retain nodes whose scrollable attribute is set to true and valid bounds cover more than 4% of the screen, deduplicate and clip these boxes, and combine them with image-based proposals to form the candidate set for manual verification. Note that the XML serves only as an annotation aid and is not required by Hagasu at inference time.

Finally, three authors of this paper manually annotate the scrollable regions using these candidates as a starting point. Specifically, two authors independently inspect each screenshot, retain valid candidates, remove incorrect ones, and draw boxes for missed regions. The third author resolves disagreements to finalize the annotations. The annotations include each region's bounding box and swipe direction. In particular, for screenshots with multiple scrollable regions, we annotate every region, including full-page containers and local components such as lists and carousels.

3.2.2 Dataset Organization and Split. As illustrated in Figure 3, each screenshot is paired with a JSON file containing the complete set of annotated bounding boxes. The dataset is represented as:

$$\mathcal{D} = \{(I_i, B_i)\}_{i=1}^{N}, \qquad B_i = \{b_{i1}, b_{i2}, \ldots, b_{im_i}\}, \tag{1}$$



(a) (b)

Fig. 3. Examples of screenshot–JSON samples in the scrollable-region dataset. Each JSON lists the complete set of annotated regions in its corresponding screenshot, with coordinates following the `[x_min, y_min, x_max, y_max]` convention.

where N is the number of samples, I_i is the i-th screenshot, m_i is its number of scrollable regions, and b_{ij} is the bounding box of the j-th region. We will further train `Hagasu` to predict the complete region set B_i for each screenshot.

The resulting dataset contains 667 screenshots, including 364 with a single annotated region and 303 with multiple regions. Following the principle of separating model training, model selection, and final evaluation [18], we randomly divide the dataset into 540 training, 60 validation, and 67 test samples.

3.3 Model Fine-tuning

Since general-purpose multimodal LLMs struggle to accurately identify scrollable regions, as shown in Table 2, we fine-tune a pretrained vision-language model on our manually collected and annotated screenshots to specialize it for scrollable-region perception. The input consists of a screenshot I_i and a fixed textual prompt asking the model to identify all scrollable regions, without requiring accessibility XML or a task-specific target. The model is supervised with the annotated region set B_i and produces a JSON object containing a variable-length list of predicted bounding boxes:

$$\hat{B}_i = f_\theta(I_i, p) = \{\hat{b}_{ij}\}_{j=1}^{\hat{m}_i}, \tag{2}$$

where f_θ denotes the fine-tuned model with parameters θ, p is the fixed textual prompt, $\hat{m}_i$ is the number of predicted regions, and $\hat{b}_{ij}$ is the bounding box of the j-th predicted region.

3.3.1 Multi-Region Sampling. During annotation, we found that many UI screens contain multiple scrollable regions. Identifying the complete region set on these screens is substantially more challenging than detecting a single region, as local scrollable regions can easily be missed. Specifically, 303 of the 667 screenshots contain multiple scrollable regions, accounting for 45.4% of the we selected screenshots. Thus, these screens reflect a common pattern in practical mobile interfaces, where a full-page container may coexist with local lists, carousels, or other scrollable components. All scrollable regions on these screens must be identified, since the task-relevant component may be located in any one of them.

To address low recall on screens with multiple scrollable regions, we therefore oversample training screenshots according to the number of annotated regions. Specifically, the number of repetitions per epoch for each training screenshot is equal to its number of annotated scrollable regions, capped at five. Table 2 in the following Section 5 shows that this reweighting effectively improves the model's recall by increasing its exposure to multi-region examples during training.

3.3.2 Two-Stage LoRA SFT. Scrollable regions lack clear visual indicators, making it difficult for models to recognize them and associate their visual appearance with swipe interaction semantics. Given the limited size of our manually collected dataset, we adopt LoRA [19] to adapt the model through a small number of trainable parameters instead of full-parameter fine-tuning. To address this challenge while making effective use of the limited data, we organize LoRA fine-tuning into two stages. The first stage strengthens visual perception of UI images, focusing on the appearance and spatial structure of UI regions. The second stage fine-tunes the language model to connect swipe interaction semantics with the visual features learned in the first stage.

Following prior work on Qwen-based GUI models [16, 25], we select Qwen3.5-9B as the backbone of `Hagasu`. Specifically, the first stage trains LoRA adapters in the visual component while keeping the original visual weights, the merger, and the language model frozen. The trained visual adapters are then merged into the backbone to initialize the second stage. In the second stage, we train newly introduced language LoRA adapters and all merger parameters while keeping the adapted visual component and the original language-model weights frozen. Both stages use LoRA with rank 16, $\alpha = 32$, and dropout 0.1, with supervision provided by screenshots paired with JSON annotations and training samples drawn using the multi-region oversampling strategy described above.

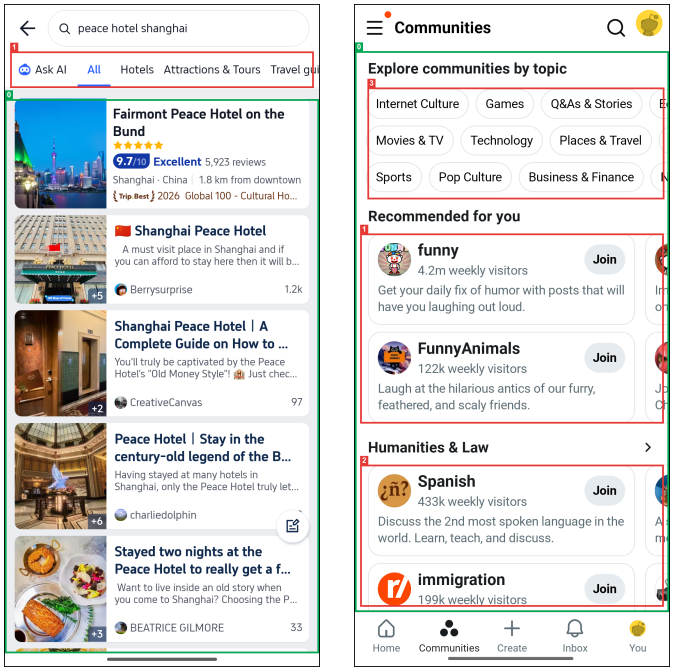
The 667 screenshots are randomly divided into 540 training, 60 validation, and 67 test samples. After oversampling, the 540 training screenshots yield 964 training entries per epoch, while the validation and test sets remain unchanged. The validation set is used for model selection, and the test set is reserved for evaluation in Section 5.

3.4 Agent Integration

3.4.1 Screenshot Annotation and Interaction Loop. We make minimal modifications to integrate `Hagasu` into various GUI agents as a non-intrusive plug-in, providing supplementary visual and textual cues for the model's automated exploration.

At each interaction step, agent captures a screenshot and passes it to `Hagasu`, which returns a JSON list of bounding boxes for the predicted scrollable regions. As shown in Figure 4, we annotate the screenshot with numbered boxes and add the following cue c to the agent's prompt: *"The screenshot has numbered boxes marking predicted scrollable regions."* Using the task instruction, annotated screenshot, and interaction history, the agent generates its next action, executes it on the device, and captures a new screenshot for subsequent planning. Algorithm 1 provides a pseudocode summary of our agent loop.

When integrated into GUI agents, `Hagasu` is called for every screenshot. Although `Hagasu` modifies the screenshot and prompt, these modifications serve only as supplementary information



(a) (b)

Fig. 4. Scrollable-region annotations predicted by Hagasu on the screenshots from Figure 3. Green and red boxes indicate vertically and horizontally scrollable regions, respectively, with region IDs at the top-left corners.

about where swipe exploration is possible. They neither force the agent to make any particular decision nor override its action policy. Therefore, these agents retain full control over whether and how to click, swipe, or perform other supported actions based on the task discription and the current interaction state.

3.4.2 Memory Management. Although this paper simplifies most GUI-agent components, such as reflection, to highlight the effect of Hagasu on planning, memory remains essential for multi-step interaction tasks. We therefore provide a simplified memory management module to support subsequent evaluation. Specifically, two memory management strategies are designed for offline and online experiments. In offline experiments, the history briefly describes the screenshots and actions from preceding steps. Since the preceding steps are deterministic, we prepared a predetermined interaction history for each step. At each step, the model receives the interaction history of all preceding steps in that task. In online experiments, the agents will generate their own interaction history after each step while performing the task. In addition to screenshot and action descriptions in the same format as in offline experiments, the agent also retains the previous screenshot and compares it with the current one to determine whether the action changed the page, helping reduce the risk of entering an infinite loop.

Algorithm 1: GUI Agent Execution with Hagasu

Input: Task T, agent prompt P, region cue c, maximum device actions *budget*

$P_{\text{aug}} \leftarrow \text{SupplementPrompt}(P, c)$;
$hist \leftarrow [\,]$; $step \leftarrow 0$;
while *true* **do**
  $S \leftarrow \text{Screenshot}()$;
  $\hat{B} \leftarrow \text{Hagasu}(S)$;
  $S_{anno} \leftarrow \text{DrawBoxes}(S, \hat{B})$;
  $act \leftarrow \text{AgentDecide}(T, S_{anno}, P_{\text{aug}}, hist, step, budget)$;
  if act = finish **then**
    return AwaitHumanJudgment;
  end
  if $step \geq budget$ **then**
    return BudgetExhausted;
  end
  ExecuteOnDevice(act);
  $step \leftarrow step + 1$;
  $S' \leftarrow \text{Screenshot}()$;
  $mem \leftarrow \text{AgentMemory}(S, S', act)$;
  $hist \leftarrow hist \parallel [mem]$;
end

4 Experiment Design

Experiments are conducted to evaluate Hagasu's swipe perception capability, its support for swipe planning, and its impact on end-to-end task completion when integrated into GUI agents. The three research questions are as follows.

- **RQ1: Scrollable Region Perception.** How accurately can Hagasu detect and localize scrollable regions in mobile UIs?
- **RQ2: Swipe-aware Planning.** How effectively does scrollable-region perception help GUI agents make correct swipe decisions?
- **RQ3: Agent Integration.** How much does Hagasu affect GUI agents' end-to-end task completion on swipe-dependent tasks, and what overhead does it introduce?

4.1 Datasets

For **RQ1**, we evaluate whether Hagasu can correctly identify all scrollable regions in a UI screenshot and accurately localize their boundaries. We use the dataset described in Section 3, containing 67 mobile UI screenshots. Each screenshot is annotated with all scrollable regions and their boundaries. Note that a screenshot may contain multiple scrollable regions. Therefore, we compare the predicted region set with the complete ground-truth set.

For **RQ2**, we construct an offline benchmark from human-recorded successful trajectories of 60 mobile GUI interaction tasks, with each trajectory containing an average of 4.1 actions and at least one swipe. Specifically, this benchmark contains 245 interaction steps, including 99 swipes (39 vertical and 60 horizontal) and 146 clicks. It evaluates whether agents can correctly decide between acting on visible content and exploring off-screen content when prompted with Hagasu's scrollable-region predictions.

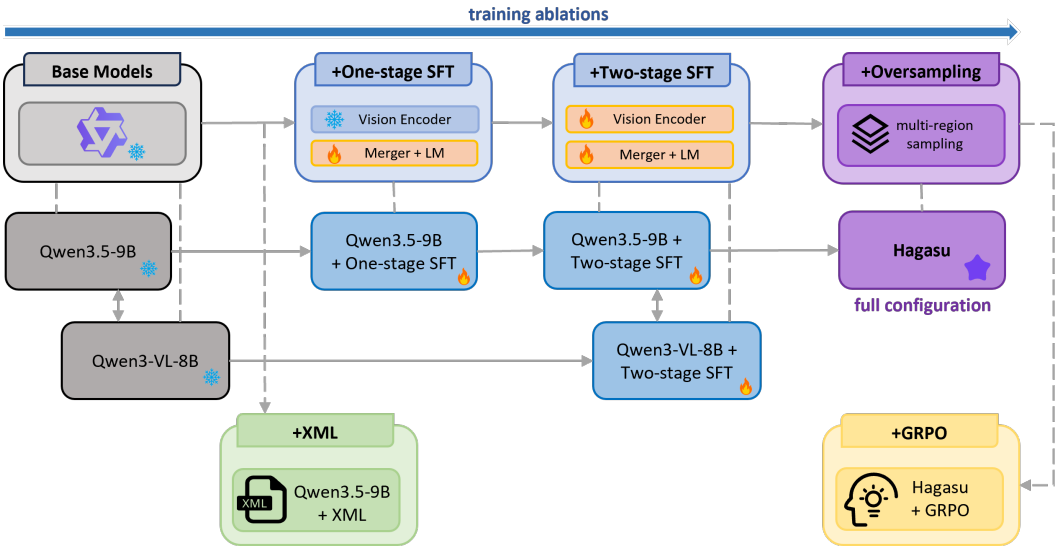


Fig. 5. Baseline and ablation configurations for RQ1. Arrows indicate the controlled change from one configuration to another.

For **RQ3**, considering execution cost and time overhead, we randomly select 18 swipe-dependent tasks across 16 Android apps from the same task pool as RQ2 for end-to-end online evaluation. Each task requires at least one swipe to reveal off-screen task-relevant content, allowing us to examine whether improved swipe planning translates into higher task completion rates. The tasks cover content discovery, media playback, product search, and travel-related information retrieval.

4.2 Baselines

For **RQ1**, we compare Hagasu with both untuned base models and several training variants to understand where the improvement in scrollable-region perception comes from. Specifically, we first evaluate two untuned baselines, *Qwen3-VL-8B* [15] and *Qwen3.5-9B* [8]. We also include *Qwen3.5-9B + XML*, where scrollable-region candidates extracted from XML are provided to the model for visual refinement. This comparison examines whether explicit structural cues can compensate for the lack of task-specific fine-tuning.

We then construct several variants of Hagasu to study its training design. *Qwen3.5-9B + one-stage SFT* directly fine-tunes the merger and language-model layers while keeping the visual encoder frozen. *Qwen3.5-9B + two-stage SFT* further introduces the visual fine-tuning stage described in Section 3. Besides, to examine the effect of the base model, we apply the same two-stage SFT procedure to Qwen3-VL-8B, yielding *Qwen3-VL-8B + two-stage SFT*. Our full Hagasu configuration further applies multi-region oversampling on top of Qwen3.5-9B with two-stage SFT. Finally, we append a GRPO stage to the full configuration to examine whether reinforcement learning provides additional gains. The relationship among these configurations is summarized in Figure 5.

For **RQ2**, we evaluate nine VLMs with different model scales and GUI capabilities: GPT-6-Luna [5], GPT-5.6-Luna [4], Seed-2.1-Pro [12], Seed-Evolving [11], Qwen3.8-27B [10], Qwen3.7-Plus [9], Qwen3.5-9B [8], DeepSeek-V4-Flash-Vision [3], and GUI-OWL-7B [52].

Each model is evaluated under three conditions: *V* stands for Vanilla, which uses the original screenshot as input. +Hagasu is abbreviated as *H*, scrollable regions predicted by Hagasu are

highlighted on the input screenshots. We further include a *G*(+Ground Truth) condition, where manually annotated scrollable regions are highlighted. This condition approximates an upper bound for region-aware planning and helps separate errors caused by Hagasu.

For **RQ3**, we select three strong-performing VLMs from RQ2 for end-to-end evaluation: Qwen3.8-27B, Qwen3.7-Plus, and GPT-6-Luna. Each model is evaluated in both *V*(Vanilla) and *H*(+Hagasu) settings. We repeat each task three times under each model and setting to account for variability in online execution.

4.3 Metrics

For **RQ1**, we evaluate two aspects of scrollable-region perception: whether Hagasu identifies the correct set of regions and how accurately it localizes their boundaries.

To evaluate *region detection capability*, we report Precision, Recall, F1, and Exact Match. Predicted and ground-truth regions are matched one-to-one within each screenshot using the Hungarian algorithm [21], with IoU ≥ 0.5 as the matching threshold. Matched predictions are counted as true positives, while unmatched predictions and ground-truth regions are counted as false positives and false negatives, respectively. Precision, Recall, and F1 are micro-averaged over the entire test set. Exact Match measures the percentage of screenshots for which the complete set of scrollable regions is correctly detected, with neither missing nor extra regions.

As for *localization accuracy*, we report set-level mIoU. For each screenshot, we perform one-to-one matching between predicted and ground-truth regions to maximize their total IoU. The summed IoU is normalized by the larger number of predicted or ground-truth regions, such that both missing and extra regions are penalized. We then average the resulting scores across all screenshots.

For **RQ2**, we report three metrics to evaluate both the benefit and potential side effects of scrollable-region cues. *Swipe Prediction Recall* is the percentage of the 99 reference swipe steps correctly predicted as swipes, measuring the agent's ability to identify when exploration is needed. *Click Prediction Recall* is the percentage of the 146 reference click steps correctly predicted as clicks, measuring whether region cues preserve normal click decisions. Finally, **Click→Swipe Rate** is the percentage of reference click steps incorrectly predicted as swipes, specifically measuring whether the cues cause agents to over-select swipe actions.

For **RQ3**, we report task success rate (SR) and σ(SR) to assess task completion and stability, with σ(SR) denoting the population standard deviation of the three round-level success rates. We compare the average number of interaction steps across all runs and across successful runs in each setting to evaluate interaction efficiency. We also report average execution time (Avg. Time) and input/output token usage (Avg. I Token and Avg. O Token) per run to measure execution cost. Token usage covers reported agent-model decision and memory generation calls and Hagasu region-annotation inference, with each component's mean rounded down to an integer.

4.4 Implementation

We collect mobile UI screens from Android devices through ADB [1], recording both the screenshot and its accessibility XML for each UI state. During annotation, XML- and image-based proposals are used only as candidate regions to reduce manual effort. Two annotators independently verify these candidates, remove invalid regions, and add missed regions, while disagreements are resolved by a third annotator. The final annotation contains the complete set of manually verified scrollable-region bounding boxes for each screenshot.

Hagasu is fine-tuned from a vision-language backbone using screenshot–region supervision. Given a screenshot, the model outputs the bounding boxes of all detected scrollable regions in JSON format.

Table 2. Comparison of Hagasu and baselines on scrollable-region detection.

Method	Precision@0.5	Recall@0.5	F1@0.5	mIoU	Exact Match@0.5
Qwen3-VL-8B	57.63	26.15	35.98	25.55	16.42
Qwen3.5-9B	58.46	29.23	38.97	31.63	23.88
Qwen3.5-9B + XML	62.30	52.05	56.72	54.30	44.68
Qwen3.5-9B + one-stage SFT	**78.00**	60.00	67.83	54.21	41.79
Qwen3.5-9B + two-stage SFT	73.68	64.62	68.85	56.24	**44.78**
Qwen3-VL-8B + two-stage SFT	70.00	59.23	64.17	50.63	32.84
Hagasu	68.46	**78.46**	**73.12**	**60.20**	43.28
Hagasu + GRPO	65.58	77.69	71.13	58.79	38.81

In the plug-in setting, an annotation service retrieves or predicts region boxes for each decision screenshot, renders them on the image, and adds a textual cue that the marked regions are scrollable. The base agent then combines this augmented observation with the task and interaction history to decide whether and how to click or swipe, while retaining control of action execution.

For **RQ3**, tasks are executed on a Xiaomi 13 smartphone running Android 16 with a screen resolution of 2400×1080 pixels. The initial state is manually restored before each run. For a given task, the instruction and action budget remain fixed across agents, settings, and repetitions. To reduce repeated annotation, Hagasu employs a persistent cache of unannotated screenshots and their predicted region boxes. When the current screenshot matches a cached image of the same dimensions with a structural similarity index (SSIM) of at least 0.90, the boxes from the most similar image are rendered on the current screenshot. Otherwise, Hagasu predicts the regions and stores the result for subsequent reuse.

5 Evaluation Result and Analysis

5.1 RQ1: Scrollable-Region Detection

Overall performance. Hagasu substantially outperforms the untuned vision-language models in scrollable-region perception. As shown in Table 2, Hagasu achieves 78.46% Recall and 73.12% F1, improving over untuned Qwen3.5-9B by 49.23 and 34.15 percentage points, respectively. Providing Qwen3.5-9B with XML-derived candidate regions improves F1 from 38.97% to 56.72%, confirming that structural cues are useful. Nevertheless, Hagasu further improves F1 to 73.12%, showing that task-specific fine-tuning is substantially more effective than directly relying on XML candidates.

Effect of training designs. Multi-region oversampling primarily improves region coverage. Removing it reduces Recall from 78.46% to 64.62% and F1 from 73.12% to 68.85%, while increasing Precision from 68.46% to 73.68%. This trade-off is consistent with the goal of oversampling: reducing missed regions at the cost of some additional false positives. The two-stage SFT schedule provides a smaller but consistent improvement over one-stage SFT, higher Recall, mIoU, and Exact Match. These results support the use of both visual fine-tuning and multi-region oversampling in Hagasu.

Other design choices. Appending GRPO to Hagasu provides no additional benefit, reducing F1 from 73.12% to 71.13% and Exact Match from 43.28% to 38.81%. This suggests that predicting floating-point coordinates for multiple regions creates a large search space that may hinder reinforcement learning convergence with limited training data. Despite the substantial improvement in region coverage, complete region-set recovery remains challenging, with Hagasu achieving an Exact Match of 43.28%.

Table 3. Offline experiment results.

Model	Swipe Prediction Recall(%) ↑			Click Prediction Recall(%) ↑			Click→Swipe Rate(%) ↓		
	V	H (ΔV)	G (ΔV; ΔH)	V	H (ΔV)	G (ΔV; ΔH)	V	H (ΔV)	G (ΔV; ΔH)
GPT-6-Luna	58.59	65.66 (+7.07)	64.65 (+6.06; −1.01)	94.52	94.52 (0.00)	91.78 (−2.74; −2.74)	1.37	2.05 (+0.68)	2.05 (+0.68; 0.00)
GPT-5.6-Luna	50.51	52.53 (+2.02)	55.56 (+5.05; +3.03)	93.84	91.78 (−2.05)	91.10 (−2.74; −0.68)	2.74	5.48 (+2.74)	4.79 (+2.05; −0.68)
Seed-2.1-Pro	51.52	55.56 (+4.04)	53.54 (+2.02; −2.02)	92.47	93.84 (+1.37)	91.78 (−0.68; −2.05)	6.16	4.79 (−1.37)	6.85 (+0.68; +2.05)
Seed-Evolving	49.49	52.53 (+3.03)	45.45 (−4.04; −7.07)	97.95	**97.95** (0.00)	**97.26** (−0.68; −0.68)	0.68	**1.37** (+0.68)	**1.37** (+0.68; 0.00)
Qwen3.8-27B	**59.60**	**66.67** (+7.07)	**65.66** (+6.06; −1.01)	95.89	95.21 (−0.68)	93.84 (−2.05; −1.37)	2.74	4.11 (+1.37)	4.79 (+2.05; +0.68)
Qwen3.7-Plus	57.58	63.64 (+6.06)	64.65 (+7.07; +1.01)	96.58	94.52 (−2.05)	95.21 (−1.37; +0.68)	2.05	3.42 (+1.37)	3.42 (+1.37; 0.00)
Qwen3.5-9B	24.24	42.42 (+18.18)	34.34 (+10.10; −8.08)	**98.63**	90.41 (−8.22)	91.10 (−7.53; +0.68)	**0.00**	7.53 (+7.53)	7.53 (+7.53; 0.00)
DeepSeek-V4	36.36	48.48 (+12.12)	46.46 (+10.10; −2.02)	95.89	95.89 (0.00)	91.78 (−4.11; −4.11)	2.05	2.05 (0.00)	5.48 (+3.42; +3.42)
GUI-OWL-7B	26.26	37.37 (+11.11)	43.43 (+17.17; +6.06)	91.10	87.67 (−3.42)	87.67 (−3.42; 0.00)	0.68	4.11 (+3.42)	2.74 (+2.05; −1.37)
Average	46.02	53.87 (+7.86)	52.64 (+6.62; −1.23)	95.21	93.53 (−1.67)	92.39 (−2.82; −1.14)	2.05	3.88 (+1.83)	4.34 (+2.28; +0.46)

[1] DeepSeek-V4 is an abbreviation for DeepSeek-V4-Flash-Vision.

5.2 RQ2: Swipe-aware Planning Improvement

Effect on swipe decisions. As shown in Table 3, adding Hagasu improves swipe prediction recall for all nine VLMs. On average, swipe prediction recall increases from 46.02% under *Vanilla* to 53.87% under *+*Hagasu, with gains ranging from 2.02 to 18.18 percentage points. The largest improvement is observed on Qwen3.5-9B (+18.18 points), followed by DeepSeek-V4-Flash-Vision (+12.12) and GUI-OWL-7B (+11.11). These results show that explicit scrollable regions consistently help VLMs recognize when a swipe is required, including models with substantially different vanilla performance.

Effect on click decisions. The improvement in swipe decisions introduces only limited interference with click decisions. While Hagasu increases average swipe recall by 7.85 percentage points, click recall decreases by only 1.68 points, from 95.21% to 93.53%. Meanwhile, the Click→Swipe rate increases modestly from 2.05% to 3.88% (+1.83 points). These results show that Hagasu makes agents substantially more likely to swipe when exploration is needed, without causing them to swipe unnecessarily.

Comparison with ground-truth regions. Replacing Hagasu predictions with ground-truth scrollable regions brings only limited additional benefit to swipe planning. On average, Swipe Recall under *+Ground Truth* is 52.64%, close to the 53.87% achieved by *+*Hagasu. This small gap suggests that Hagasu's predicted regions already capture most of the scrollability information that is useful for swipe decisions. At the same time, the limited additional benefit from ground-truth regions indicates that scrollable-region perception is not the only factor affecting swipe planning. Even

Table 4. Online task performance and execution cost with and without Hagasu.

Agent	Mode	SR (%)	σ(SR)	Total/Succ. (steps/task)	Time (s/task)	Input Token (/task)	Output Token (/task)
Qwen3.8-27B	V	70.37	5.24	10.93 / 10.84	85.84	75,823	822
	H	88.89 (+18.52)	4.54	10.02 / 9.29	129.77	69,472 + 2,261	757 + 531
Qwen3.7-Plus	V	74.07	2.62	11.06 / 9.53	88.79	76,800	909
	H	88.89 (+14.81)	4.54	10.07 / 9.50	122.19	69,847 + 1,805	816 + 418
GPT-6-Luna	V	50.00	12.00	10.11 / 9.52	97.41	112,739	1,058
	H	70.37 (+20.37)	6.93	11.59 / 10.79	168.48	128,915 + 2,517	1,119 + 598

[1] Total/Succ. reports mean steps over all tasks / successful tasks.
[2] σ(SR) is the population standard deviation of three round-level SRs. Parenthesized SR gains for +Hagasu over *Vanilla* use original success counts.
[3] Token usage is reported separately for the agent and Hagasu, shown as <agent + Hagasu >: agent tokens cover decision-making and memory generation, while Hagasu tokens correspond to region-annotation inference.

with accurate region cues, agents still need to determine how to incorporate them into task-level action selection.

5.3 RQ3: Plug-in Effectiveness, Overhead, and Side Effects

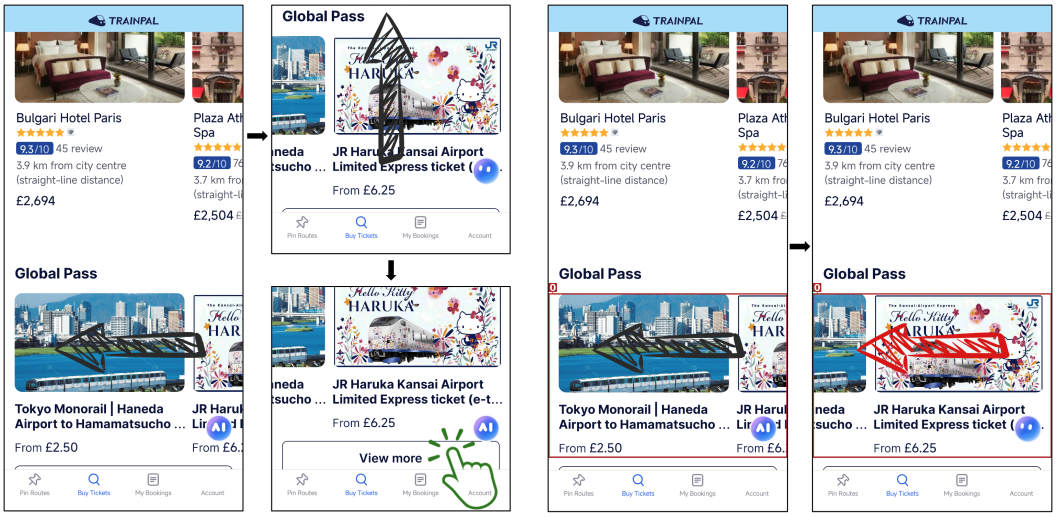
RQ3 evaluates how Hagasu affects the end-to-end task automation performance of GUI agents. This experiment examines whether scrollable-region perception can lead to higher end-to-end task success and assesses the additional execution cost. The three VLMs with the highest swipe prediction recall in **RQ2**'s +Hagasu setting are selected for end-to-end evaluation. GUI agents powered by these VLMs are then evaluated under *Vanilla* and +Hagasu on the 18 swipe-dependent tasks, with each task run three times per agent and setting and success assessed manually for each run. Table 4 reports task success rate (SR) and its standard deviation across rounds (σ(SR)), average interaction steps for all runs and for successful runs, and per-run averages of execution time and input/output token usage.

Task completion and consistency. All three agents achieve higher task success rates under +Hagasu on swipe-dependent tasks, with the mean rising from 64.81% to 82.72% (+17.90 percentage points). This agreement with **RQ2** suggests that region cues remain useful when swipe decisions are embedded in complete task trajectories. GPT-6-Luna shows the largest gain in task success rate, rising from 50.00% to 70.37%, while both Qwen agents reach 88.89%, showing that scrollable-region perception benefits even agents with stronger baseline performance.

Moreover, Hagasu reduces the standard deviation of task success rates across evaluation rounds for Qwen3.8-27B and GPT-6-Luna, particularly the latter with its lower baseline success rate, indicating more consistent task completion across repeated runs. Nevertheless, we acknowledge that Hagasu's prediction errors can contribute to task failures. A representative case is discussed in Section 6.1.

Interaction efficiency. Given GUI agents' ability to recover from suboptimal actions during task execution, we evaluate whether the proactive exploration encouraged by Hagasu reduces the number of interactions required to complete a task. Overall, +Hagasu reduces the mean number of interactions per task, with the effect varying across agents: Qwen3.8-27B and Qwen3.7-Plus complete more tasks with fewer interactions, whereas GPT-6-Luna follows longer trajectories.

Execution cost. Adding Hagasu naturally introduces additional execution overhead, increasing active execution time by 49.46 seconds per run on average, with increases ranging from 33.40 to



(a) Vanilla exploration (b) Exploration with Hagasu

Fig. 6. Comparison of Qwen3.8-27B's exploration in TrainPal: (a) Vanilla exploration reveals *View more* through page scrolling, whereas (b) exploration with Hagasu remains confined to the carousel. Arrows indicate swipe actions, and red boxes mark scrollable regions predicted by Hagasu.

71.07 seconds across agents. This overhead increases the time required to obtain a task outcome and may reduce the number of attempts possible under a fixed time budget. These measurements already account for cache reuse, with hit rates ranging from 62.30% to 70.34% among calls carrying cache metadata.

However, Hagasu does not increase token consumption for the Qwen-based agents. Their lower token usage, including region-annotation inference, is consistent with the shorter interaction trajectories enabled by more targeted exploration, suggesting that improved exploration efficiency can also reduce the token cost of agent decision-making and reflection. Overall, while Hagasu improves task success at the cost of additional execution time, its more accurate exploration can reduce token consumption for capable agents.

6 Discussion

This section first uses a case study to show that, although scrollable-region perception has proven valuable for GUI agents, integrating it through supplementary cues has inevitable limitations. The case illustrates how Hagasu can degrade an agent's task performance. It then considers threats to validity arising from manual annotation and variations in agent performance.

6.1 Case Study: Towards Integrated Perception and Planning

In RQ3, we observe that although Hagasu is designed to preserve agents' decision-making autonomy, its predictions can still misguide their exploration. To better understand this phenomenon, we analyze a failed execution run on TrainPal's homepage. The task requires locating *KORAIL PASS* through the *Global Pass* section on TrainPal's homepage. In a **+Hagasu** run, the Qwen3.8-27B-based agent fails to locate *KORAIL PASS*, illustrating how incomplete region cues can interfere with its exploration.

Figure 6(a) shows the carousel endpoints and the route to the full catalogue for Vanilla exploration. Neither end of the carousel displays *KORAIL PASS*. In a successful Vanilla run, the agent swipes up on the page to reveal *View more*, which is initially concealed beneath the bottom navigation bar. Selecting *View more* opens the full *Global Pass* catalogue, which lists the available passes and tickets by category. Within the catalogue, the agent swipes left across the category bar, selects *South Korea*, and opens *KORAIL PASS*.

In the illustrated steps of the failed run, `Hagasu` highlights the horizontally scrollable preview while omitting the surrounding vertically scrollable page. The *View more* label remains concealed beneath the bottom navigation bar, therefore reaching the full catalogue requires further vertical page scrolling. As shown in Figure 6(b), the agent swipes left from the highlighted carousel's left-end view to its right-end view. This transition changes the carousel's visible content, but further leftward swipes at the right boundary produce no visible change and do not reveal *KORAIL PASS*. The agent continues exploring the carousel instead of scrolling the page to expose *View more*, and ultimately returns `finish` on the homepage without locating the requested pass.

This case indicates that preserving an agent's decision-making autonomy does not fully prevent incomplete perception from misleading its planning. Although `Hagasu` only provides supplementary guidance without enforcing any action, the agent may still rely too heavily on these cues and fail to revise its search strategy after repeated unsuccessful interactions. This suggests future research on internalizing scrollable-region perception into GUI agents through large-scale joint training with task planning, so that agents can systematically learn when to explore, where to explore, and when to stop exploring.

6.2 Threats to Validity

Annotation subjectivity. Since scrollable regions may lack clear visual boundaries, manual annotations are susceptible to omissions and errors. Local or nested regions can be overlooked, and annotators may disagree on their boundaries, affecting both training supervision and the reference annotations used in RQ1. Although XML-derived candidates provide an initial set of regions for inspection, the candidate set may omit scrollable regions or include non-scrollable ones. To mitigate this threat, two authors independently inspect each screenshot, remove invalid candidates, and add missed regions. A third author resolves disagreements to finalize the annotations. Although some annotation noise may remain, candidate-assisted inspection helps identify potential regions, while independent checks and disagreement resolution reduce reliance on individual judgments.

Stability of effectiveness. Changes in UI states and variations in agent behavior may affect the observed performance gains from `Hagasu`. The same task may encounter different interface states or action sequences across executions, so a single run may overestimate or underestimate the benefit of the plug-in. We acknowledge this threat and seek to mitigate it by evaluating diverse test tasks with multiple agents. This broader evaluation reduces dependence on a particular task or agent's decision patterns. In RQ3, `Hagasu` improves the task success rate of all three evaluated agents. Moreover, each task in RQ3 is executed three times, and we report the mean within-agent population standard deviation of round-level task success rates (5.33 percentage points under **+Hagasu**, obtained by averaging the three agents' unrounded standard deviations over the three evaluation rounds; Table 4). These repeated runs allow us to assess whether the observed gains persist across executions and quantify the remaining variation.

7 Related Work

Interactive region perception. Identifying and localizing interactive regions is fundamental to GUI task automation. Structured approaches use UI hierarchies and element attributes to expose interaction targets [17, 42], while visual approaches ground actions directly in screenshots. ScreenAI [14] studies screen understanding and UI element annotation, and SeeClick [16] develops GUI grounding through visual pretraining. Building on visual localization, ShowUI [25], OS-Atlas [45], and UIPro [22] further connect grounding with GUI action prediction. ScreenSpot-Pro [23] evaluates these localization capabilities in high-resolution professional interfaces.

Beyond locating interaction targets, existing work also investigates what interactions UI regions support. Schoop et al. [35] predict whether users perceive UI elements as tappable, while Ferret-UI [53] supports UI function inference and goal-oriented interaction conversations. OmniParser [28] combines interactable-element detection with functional descriptions to support action grounding. Complementing such perception methods, Set-of-Mark prompting [50] overlays marks on image regions so that a multimodal model can refer to them explicitly. These methods improve the localization, functional understanding, and explicit referencing of visible interaction targets. However, none specifically focuses on identifying scrollable containers that can reveal content beyond the current viewport.

GUI exploration and planning. Existing work guides GUI exploration to improve testing efficiency. Stoat [37] uses heuristic exploration strategies and GUI models to guide test generation. LLM-Droid [40] uses general-purpose LLMs to direct existing testing tools toward additional functionality when coverage growth slows. Guardian [31] uses runtime action constraints and replanning to guide exploration toward UI states relevant to testing objectives. These approaches focus on selecting and adjusting exploration actions to improve testing efficiency, rather than explicitly learning the interaction semantics that determine which UI regions support scrolling.

Recent GUI agents also improve planning through interaction history, reflection, and proactive exploration [24, 26, 43]. In particular, MobileUse [24] combines hierarchical reflection with self-planned exploration to build knowledge of unfamiliar environments. These mechanisms address how agents accumulate and use experience to guide subsequent actions. However, for complex apps, building such knowledge can require extensive preliminary exploration without guaranteeing useful guidance for a given task. `Hagasu` instead provides scrollable-region cues directly from the current screenshot and is shown to be effective with modest overhead.

8 Conclusion

Motivated by the huge gap between agents' GUI and CLI task performance, this paper identifies invisible content as a key bottleneck for GUI tasks. When the correct target is hidden, agents often click an incorrect visible target instead of exploring. Built on a manually annotated dataset, `Hagasu` provides non-intrusive scrollable-region cues that support hidden-area exploration while preserving GUI agents' decision-making autonomy. Experiments demonstrate that such scrollable-region prompts improve swipe selection accuracy by 17.1% and result in a 27.6% increase in the end-to-end task completion rate. These findings suggest that making exploration opportunities explicit is a practical direction for improving GUI agent harnesses under partial observability.

9 DATA AVAILABILITY

All of the source code, dataset, and experimental records for `Hagasu` are available at https://figshare.com/s/c39e36c1c8fb0b48a1b6.

References

[1] Accessed: 2026. *Android Debug Bridge (adb).* https://developer.android.com/tools/adb

[2] Accessed: 2026. *App Discovery and Ranking.* https://support.google.com/googleplay/android-developer/answer/9958766

[3] Accessed: 2026. *DeepSeek-V4-Flash-Vision-Exp Release.* https://api-docs.deepseek.com/updates/#deepseek-v4-flash-vision-exp-release

[4] Accessed: 2026. *GPT-5.6 Luna.* https://developers.openai.com/api/docs/models/gpt-5.6-luna

[5] Accessed: 2026. *GPT-6 Luna.* https://developers.openai.com/api/docs/models/gpt-6-luna

[6] Accessed: 2026. *Introducing GPT-5.4.* https://openai.com/index/introducing-gpt-5-4/

[7] Accessed: 2026. *Lists—Material Design.* https://m1.material.io/components/lists.html

[8] Accessed: 2026. *Qwen3.5-9B.* https://huggingface.co/Qwen/Qwen3.5-9B

[9] Accessed: 2026. *Qwen3.7-Plus: Multimodal Agent Intelligence.* https://qwen.ai/blog?id=qwen3.7-plus

[10] Accessed: 2026. *Qwen3.8-27B.* https://huggingface.co/Qwen/Qwen3.8-27B

[11] Accessed: 2026. *Seed-Evolving.* https://docs.volcengine.com/docs/ark/latest-model?lang=zh

[12] Accessed: 2026. *Seed2.1.* https://seed.bytedance.com/en/seed2_1

[13] Accessed: 2026. *UI Automator: AccessibilityNodeInfoDumper.* https://android.googlesource.com/platform/frameworks/testing/+/master/uiautomator/library/core-src/com/android/uiautomator/core/AccessibilityNodeInfoDumper.java

[14] Gilles Baechler, Srinivas Sunkara, Maria Wang, Fedir Zubach, Hassan Mansoor, Vincent Etter, Victor Carbune, Jason Lin, Jindong Chen, and Abhanshu Sharma. 2024. ScreenAI: A Vision-Language Model for UI and Infographics Understanding. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*. ijcai.org, 3058–3068.

[15] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibo Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yuanzhi Zhu, and Ke Zhu. 2025. Qwen3-VL Technical Report. *CoRR* abs/2511.21631 (2025).

[16] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. 2024. SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 9313–9332.

[17] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems*.

[18] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press.

[19] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.

[20] Quyu Kong, Xu Zhang, Zhenyu Yang, Nolan Gao, Chen Liu, Panrong Tong, Chenglin Cai, Hanzhang Zhou, Jianan Zhang, Liangyu Chen, Zhidan Liu, Steven Hoi, and Yue Wang. 2026. MobileWorld: Benchmarking Autonomous Mobile Agents in Agent-User Interactive and MCP-Augmented Environments. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 6142–6167.

[21] Harold W. Kuhn. 1955. The Hungarian Method for the Assignment Problem. *Naval Research Logistics Quarterly* 2, 1-2 (1955), 83–97.

[22] Hongxin Li, Jingran Su, Jingfan Chen, Zheng Ju, Yuntao Chen, Qing Li, and Zhaoxiang Zhang. 2025. UIPro: Unleashing Superior Interaction Capability for GUI Agents. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 1613–1623.

[23] Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. 2025. ScreenSpot-Pro: GUI Grounding for Professional High-Resolution Computer Use. In *Proceedings of the 33rd ACM International Conference on Multimedia*. Association for Computing Machinery, 8778–8786.

[24] Ning Li, Xiangmou Qu, Jiamu Zhou, Muning Wen, Kounianhua Du, Xingyu Lou, Qiuying Peng, Jun Wang, and Weinan Zhang. 2026. MobileUse: A Hierarchical Reflection-Driven GUI Agent for Autonomous Mobile Operation. *Advances in Neural Information Processing Systems* 38 (2026), 40361–40388.

[25] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Stan Weixian Lei, Lijuan Wang, and Mike Zheng Shou. 2025. ShowUI: One Vision-Language-Action Model for GUI Visual Agent. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 19498–19508.

[26] Tao Liu, Chongyu Wang, Rongjie Li, Yingchen Yu, Xuming He, and Song Bai. 2026. Gui-rise: Structured reasoning and history summarization for gui navigation. *Advances in Neural Information Processing Systems* 38 (2026), 165132–165156.

[27] Quanfeng Lu, Wenqi Shao, Zitao Liu, Lingxiao Du, Fanqing Meng, Boxuan Li, Botong Chen, Siyuan Huang, Kaipeng Zhang, and Ping Luo. 2025. GUIOdyssey: A Comprehensive Dataset for Cross-App GUI Navigation on Mobile Devices. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 22404–22414.

[28] Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. 2024. OmniParser for Pure Vision Based GUI Agent. *CoRR* abs/2408.00203 (2024).

[29] Ken Pfeuffer and Yang Li. 2018. Analysis and Modeling of Grid Performance on Touchscreen Mobile Devices. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems (CHI '18)*. Association for Computing Machinery, Article 288, 12 pages.

[30] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, Wanjun Zhong, Kuanye Li, Jiale Yang, Yu Miao, Woyu Lin, Longxiang Liu, Xu Jiang, Qianli Ma, Jingyu Li, Xiaojun Xiao, Kai Cai, Chuang Li, Yaowei Zheng, Chaolin Jin, Chen Li, Xiao Zhou, Minchao Wang, Haoli Chen, Zhaojian Li, Haihua Yang, Haifeng Liu, Feng Lin, Tao Peng, Xin Liu, and Guang Shi. 2025. UI-TARS: Pioneering Automated GUI Interaction with Native Agents. *CoRR* abs/2501.12326 (2025).

[31] Dezhi Ran, Hao Wang, Zihe Song, Mengzhou Wu, Yuan Cao, Ying Zhang, Wei Yang, and Tao Xie. 2024. Guardian: A Runtime Framework for LLM-Based UI Exploration. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 958–970.

[32] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. 2025. AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents. In *The Thirteenth International Conference on Learning Representations*.

[33] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy P. Lillicrap. 2023. AndroidInTheWild: A Large-Scale Dataset for Android Device Control. In *Advances in Neural Information Processing Systems*.

[34] Katharina Reinecke, Tom Yeh, Luke Miratrix, Rahmatri Mardiko, Yuechen Zhao, Jenny Liu, and Krzysztof Z. Gajos. 2013. Predicting Users' First Impressions of Website Aesthetics with a Quantification of Perceived Visual Complexity and Colorfulness. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '13)*. Association for Computing Machinery, 2049–2058.

[35] Eldon Schoop, Xin Zhou, Gang Li, Zhourong Chen, Björn Hartmann, and Yang Li. 2022. Predicting and Explaining Mobile UI Tappability with Vision Modeling and Saliency Analysis. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*.

[36] Peter Shaw, Mandar Joshi, James Cohan, Jonathan Berant, Panupong Pasupat, Hexiang Hu, Urvashi Khandelwal, Kenton Lee, and Kristina Toutanova. 2023. From Pixels to UI Actions: Learning to Follow Instructions via Graphical User Interfaces. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 34354–34370.

[37] Ting Su, Guozhu Meng, Yuting Chen, Ke Wu, Weiming Yang, Yao Yao, Geguang Pu, Yang Liu, and Zhendong Su. 2017. Guided, Stochastic Model-Based GUI Testing of Android Apps. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*. 245–256.

[38] Fei Tang, Zhangxuan Gu, Zhengxi Lu, Xuyang Liu, Shuheng Shen, Changhua Meng, Wen Wang, Wenqi Zhang, Yongliang Shen, Weiming Lu, Jun Xiao, and Yueting Zhuang. 2026. GUI-G^2: Gaussian Reward Modeling for GUI Grounding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 33214–33222.

[39] Venus Team, Changlong Gao, Zhangxuan Gu, Yulin Liu, Xinyu Qiu, Shuheng Shen, Yue Wen, Tianyu Xia, Zhenyu Xu, Zhengwen Zeng, Beitong Zhou, Xingran Zhou, Weizhi Chen, Sunhao Dai, Jingya Dou, Yichen Gong, Yuan Guo, Zhenlin Guo, Feng Li, Qian Li, Jinzhen Lin, Yuqi Zhou, Linchao Zhu, Liang Chen, Zhenyu Guo, Changhua Meng, and Weiqiang Wang. 2026. UI-Venus-1.5 Technical Report. *CoRR* abs/2602.09082 (2026).

[40] Chenxu Wang, Tianming Liu, Yanjie Zhao, Minghui Yang, and Haoyu Wang. 2025. LLMDroid: Enhancing Automated Mobile App GUI Testing Coverage with Large Language Model Guidance. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 1001–1022.

[41] Junyang Wang, Haiyang Xu, Jiabo Ye, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. 2024. Mobile-Agent: Autonomous Multi-Modal Mobile Device Agent with Visual Perception. *CoRR* abs/2401.16158 (2024). Presented at the ICLR 2024 Workshop on Large Language Model (LLM) Agents.

[42] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. 2024. AutoDroid: LLM-powered Task Automation in Android. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. 543–557.

[43] Penghao Wu, Shengnan Ma, Bo Wang, Jiaheng Yu, Lewei Lu, and Ziwei Liu. 2025. GUI-Reflection: Empowering Multimodal GUI Models with Self-Reflection Behavior. In *Advances in Neural Information Processing Systems*.

[44] Qianhui Wu, Kanzhi Cheng, Rui Yang, Chaoyun Zhang, Jianwei Yang, Huiqiang Jiang, Jian Mu, Baolin Peng, Bo Qiao, Reuben Tan, Si Qin, Lars Liden, Qingwei Lin, Huan Zhang, Tong Zhang, Jianbing Zhang, Dongmei Zhang, and Jianfeng Gao. 2025. GUI-Actor: Coordinate-Free Visual Grounding for GUI Agents. In *Advances in Neural Information Processing Systems*.

[45] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. 2025. OS-ATLAS: Foundation Action Model for Generalist GUI Agents. In *The Thirteenth International Conference on Learning Representations*.

[46] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems*.

[47] Yifan Xu, Xiao Liu, Xueqiao Sun, Siyi Cheng, Hao Yu, Hanyu Lai, Shudan Zhang, Dan Zhang, Jie Tang, and Yuxiao Dong. 2025. AndroidLab: Training and Systematic Benchmarking of Android Autonomous Agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2144–2166.

[48] John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., 50528–50652.

[49] John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. 2023. InterCode: Standardizing and Benchmarking Interactive Coding with Execution Feedback. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 23826–23854.

[50] Jianwei Yang, Hao Zhang, Feng Li, Xueyan Zou, Chunyuan Li, and Jianfeng Gao. 2023. Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V. *CoRR* abs/2310.11441 (2023).

[51] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*.

[52] Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Zhaoqing Zhu, Ziwei Zheng, Feiyu Gao, Junjie Cao, Zhengxi Lu, Jitong Liao, Qi Zheng, Fei Huang, Jingren Zhou, and Ming Yan. 2025. Mobile-Agent-v3: Fundamental Agents for GUI Automation. *CoRR* abs/2508.15144 (2025).

[53] Keen You, Haotian Zhang, Eldon Schoop, Floris Weers, Amanda Swearngin, Jeffrey Nichols, Yinfei Yang, and Zhe Gan. 2024. Ferret-UI: Grounded Mobile UI Understanding with Multimodal LLMs. In *Computer Vision – ECCV 2024*. Springer Nature Switzerland, 240–255.

[54] Chi Zhang, Zhao Yang, Jiaxuan Liu, Yanda Li, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. 2025. AppAgent: Multimodal Agents as Smartphone Users. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–20.

[55] Hanzhang Zhou, Xu Zhang, Panrong Tong, Jianan Zhang, Liangyu Chen, Quyu Kong, Chenglin Cai, Chen Liu, Yue Wang, Jingren Zhou, and Steven Hoi. 2025. MAI-UI Technical Report: Real-World Centric Foundation GUI Agents. *CoRR* abs/2512.22047 (2025).